DL2

Classification using Deep neural network (Any One from the following)

1. Multiclass classification using Deep Neural Networks: Example: Use the OCR letter recognition dataset https://archive.ics.uci.edu/ml/datasets/letter+recognition

2. Binary classification using Deep Neural Networks Example: Classify movie reviews into positive" reviews and "negative" reviews, just based on the text content of the reviews. Use IMDB dataset

```
# ========================
# 1. IMPORT LIBRARIES
# ========================
import numpy as np                  # for numerical operations
import pandas as pd                 # for handling dataset (CSV)
import matplotlib.pyplot as plt     # for plotting graphs
import re                           # for text cleaning using regex

from sklearn.model_selection import train_test_split   # for splitting data
from sklearn.feature_extraction.text import TfidfVectorizer  # for text to numeric

from tensorflow.keras.models import Sequential        # for building NN model
from tensorflow.keras.layers import Dense             # for adding layers


# ========================
# 2. LOAD DATASET (SAFE)
# ========================
# Load CSV file safely (ignore errors if any bad lines)
```

```
df = pd.read_csv(
    "IMDB_Dataset.csv",
    encoding_errors='ignore',
    on_bad_lines='skip',
    engine='python'
)

print("Dataset Loaded ✅ ")
print(df.head())        # show first 5 rows
print(df.columns)       # show column names


# ==========================
# 3. REMOVE NULL VALUES
# ==========================
df = df.dropna()   # remove missing values

print(df['sentiment'].unique())   # check unique values
print(df['sentiment'].isnull().sum())  # check null values count

df = df.dropna()   # again ensure no null values


# ==========================
# 4. VERIFY DATA
# ==========================
print(df['sentiment'].unique())   # should be ['positive','negative']
print(df.isnull().sum())          # all values should be 0
```

```
# ==========================
# 5. FIX COLUMN NAMES
# ==========================
# Rename columns for consistency
df.columns = ['review', 'sentiment']


# ==========================
# 6. CLEAN TEXT
# ==========================
def clean_text(text):
    text = str(text)                    # convert to string
    text = text.lower()                 # convert to lowercase
    text = re.sub(r'<.*?>', '', text)    # remove HTML tags
    text = re.sub(r'[^a-zA-Z ]', '', text) # remove special characters/numbers
    return text

# Apply cleaning to all reviews
df['review'] = df['review'].apply(clean_text)


# ==========================
# 7. CONVERT LABELS
# ==========================
# Convert 'positive' → 1 and 'negative' → 0
df['sentiment'] = df['sentiment'].map({'positive': 1, 'negative': 0})

print("\nSentiment Count:")
```

```
print(df['sentiment'].value_counts())   # check distribution


# ==========================
# 8. TF-IDF VECTORIZATION
# ==========================
# Convert text into numerical vectors
vectorizer = TfidfVectorizer(max_features=5000)

X = vectorizer.fit_transform(df['review']).toarray()  # input features
y = df['sentiment']                                   # output labels


# ==========================
# 9. TRAIN TEST SPLIT
# ==========================
# Split dataset into training (80%) and testing (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# ==========================
# 10. BUILD MODEL
# ==========================
model = Sequential()   # create neural network model

# Input layer + hidden layer
model.add(Dense(128, activation='relu', input_shape=(X_train.shape[1],)))
```

```
# Second hidden layer
model.add(Dense(64, activation='relu'))

# Output layer (1 neuron for binary classification)
model.add(Dense(1, activation='sigmoid'))

# Compile model (define optimizer, loss, accuracy)
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()   # display model structure



# ========================
# 11. TRAIN MODEL
# ========================
# Train the model using training data
history = model.fit(
    X_train, y_train,
    epochs=10,            # number of iterations
    batch_size=32,        # batch size
    validation_data=(X_test, y_test)  # validation
)
```

```
# ==========================
# 12. PLOT GRAPH
# ==========================
# Plot training and validation accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title("Accuracy Graph")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()


# ==========================
# 13. FINAL ACCURACY
# ==========================
# Evaluate model performance on test data
loss, accuracy = model.evaluate(X_test, y_test)

print("\nFinal Accuracy:", accuracy)
```

🔹 1. IMPORT LIBRARIES

import numpy as np

👉 Used for numerical operations (arrays, calculations).

import pandas as pd

👉 Used to load and handle dataset (CSV file).

import matplotlib.pyplot as plt

👉 Used to plot graphs (accuracy graph).

import re

👉 Used for text cleaning (removing HTML, symbols).

from sklearn.model_selection import train_test_split

👉 Used to divide dataset into training and testing data.

from sklearn.feature_extraction.text import TfidfVectorizer

👉 Converts text data into numbers (important for ML model).

from tensorflow.keras.models import Sequential

👉 Used to create neural network model layer by layer.

from tensorflow.keras.layers import Dense

👉 Used to add layers (neurons) in neural network.

🔹 2. LOAD DATASET

df = pd.read_csv(...)

👉 Reads IMDB dataset from CSV file into dataframe.

- encoding_errors='ignore' → ignores encoding issues
- on_bad_lines='skip' → skips corrupted rows
- engine='python' → safe reading

print("Dataset Loaded ✅")

👉 Just confirmation message.

print(df.head())

👉 Shows first 5 rows of dataset.

```
print(df.columns)
```

👉 Shows column names (review, sentiment).

🔹 3. REMOVE NULL VALUES

```
df = df.dropna()
```

👉 Removes missing values from dataset.

```
print(df['sentiment'].unique())
```

👉 Shows unique values → positive, negative.

```
print(df['sentiment'].isnull().sum())
```

👉 Checks if any null values exist.

```
df = df.dropna()
```

👉 Ensures dataset is clean.

🔹 4. VERIFY DATA

```
print(df['sentiment'].unique())
```

👉 Confirms only two classes exist.

```
print(df.isnull().sum())
```

👉 Confirms no missing values.

🔹 5. FIX COLUMN NAMES

```
df.columns = ['review', 'sentiment']
```

👉 Renames columns properly for easy use.

🔹 6. CLEAN TEXT

```
def clean_text(text):
```

👉 Function to clean review text.

```
text = str(text)
```

👉 Converts data to string.

```
text = text.lower()
```

👉 Converts all text to lowercase.

```
text = re.sub(r'<.*?>', '', text)
```

👉 Removes HTML tags.

```
text = re.sub(r'[^a-zA-Z ]', '', text)
```

👉 Removes special characters, numbers.

```
return text
```

👉 Returns cleaned text.

```
df['review'] = df['review'].apply(clean_text)
```

👉 Applies cleaning to all reviews.

🔹 7. CONVERT LABELS

```
df['sentiment'] = df['sentiment'].map({'positive': 1, 'negative': 0})
```

👉 Converts text labels into numbers:

- positive → 1
- negative → 0

```
print(df['sentiment'].value_counts())
```

👉 Shows count of positive & negative reviews.

🔹 8. TF-IDF VECTORIZATION

```
vectorizer = TfidfVectorizer(max_features=5000)
```

👉 Converts text into numbers using TF-IDF.
👉 Keeps only top 5000 important words.

```
X = vectorizer.fit_transform(df['review']).toarray()
```

👉 Converts reviews into numerical feature matrix.

```
y = df['sentiment']
```

👉 Target variable (output).

🔹 9. TRAIN TEST SPLIT

X_train, X_test, y_train, y_test = train_test_split(...)

👉 Splits data:

- 80% → training
- 20% → testing

👉 random_state=42 ensures same split every time.

🔹 10. BUILD MODEL

model = Sequential()

👉 Creates empty neural network.

model.add(Dense(128, activation='relu', input_shape=(X_train.shape[1],)))

👉 First layer:

- 128 neurons
- ReLU activation
- Input size = number of features

model.add(Dense(64, activation='relu'))

👉 Second hidden layer with 64 neurons.

model.add(Dense(1, activation='sigmoid'))

👉 Output layer:

- 1 neuron
- Sigmoid → gives output between 0 and 1

model.compile(...)

👉 Configures model:

- optimizer = adam
- loss = binary_crossentropy
- metric = accuracy

model.summary()

👉 Displays model structure.

🔹 11. TRAIN MODEL

history = model.fit(...)

👉 Trains the model:

- epochs=10 → model runs 10 times
- batch_size=32 → processes 32 samples at once
- validation → checks performance on test data

🔹 12. PLOT GRAPH

plt.plot(history.history['accuracy'], ...)

👉 Plots training accuracy.

plt.plot(history.history['val_accuracy'], ...)

👉 Plots validation accuracy.

👉 Helps to see model performance.

🔹 13. FINAL ACCURACY

loss, accuracy = model.evaluate(X_test, y_test)

👉 Tests model on unseen data.

print("Final Accuracy:", accuracy)

👉 Displays final accuracy.

✅ FINAL VIVA SUMMARY (VERY IMPORTANT)

If examiner asks **"Explain your project in short"**, say this:

👉
"This project performs sentiment analysis on IMDB movie reviews.
First, we clean the text data and convert it into numerical form using TF-IDF.

Then we train a neural network model with two hidden layers.
Finally, we evaluate the model accuracy and visualize performance using graphs.”

🔹 DRY RUN WITH SIMPLE EXAMPLE

Assume dataset:

```
review                    sentiment
-----------------------------------------
"Movie was awesome!"        positive
"Worst film ever"           negative
```

🔹 STEP 1: LOAD DATA

df = pd.read_csv(...)

👉 Data loaded into dataframe:

review	**sentiment**
Movie was awesome!	positive
Worst film ever	negative

🔹 STEP 2: CLEAN TEXT

df['review'] = df['review'].apply(clean_text)

Processing each review:

👉 "Movie was awesome!"

- lowercase → "movie was awesome!"
- remove symbols → "movie was awesome"

👉 "Worst film ever"

- lowercase → "worst film ever"

✓ Final cleaned data:

review	sentiment
movie was awesome	positive
worst film ever	negative

🔹 STEP 3: CONVERT LABELS

```
df['sentiment'] = df['sentiment'].map({'positive': 1, 'negative': 0})
```

✓ Converted:

review	sentiment
movie was awesome	1
worst film ever	0

🔹 STEP 4: TF-IDF VECTORIZATION

```
vectorizer.fit_transform(df['review'])
```

👉 Vocabulary created:

['movie', 'was', 'awesome', 'worst', 'film', 'ever']

👉 Convert text → numbers:

review	movie	was	awesome	worst	film	ever
movie was awesome	1	1	1	0	0	0
worst film ever	0	0	0	1	1	1

👉 This becomes **X (input features)**

👉 y = [1, 0]

🔹 STEP 5: TRAIN-TEST SPLIT

```
train_test_split(...)
```

👉 Suppose:

- Training → 1 row
- Testing → 1 row

Example:

```
X_train = [movie was awesome]
y_train = [1]

X_test = [worst film ever]
y_test = [0]
```

🔹 STEP 6: MODEL BUILDING

model = Sequential()

👉 Neural network created.

Structure:

Input Layer → 128 neurons → 64 neurons → Output(1)

🔹 STEP 7: TRAINING

model.fit(...)

👉 Model learns patterns:

- Words like “awesome” → positive
- Words like “worst” → negative

👉 After multiple epochs, weights are adjusted.

🔹 STEP 8: PREDICTION (IMPORTANT)

Suppose new review:

"This movie is awesome"

👉 Clean → "this movie is awesome"

👉 Convert using TF-IDF → numeric vector

👉 Model predicts:

output = 0.92

👉 Since > 0.5 → **Positive (1)**

🔹 STEP 9: ACCURACY CHECK

model.evaluate(X_test, y_test)

👉 Model compares predicted vs actual:

Actual	Predicted
0	0

👉 Accuracy = 100% (for this small example)

🔹 STEP 10: GRAPH

plt.plot(...)

👉 Graph shows:

- Training accuracy ↑
- Validation accuracy ↑

👉 Helps check overfitting.

✅ FINAL DRY RUN SUMMARY (FOR EXAMINER)

Say this confidently:

👉
"First, the dataset is loaded and cleaned by removing special characters.
Then, text data is converted into numerical form using TF-IDF.
Next, the dataset is split into training and testing.
A neural network model is trained using training data.
Finally, the model predicts sentiment and accuracy is calculated using test data."

🔥 PRO TIP (VERY IMPORTANT)

If examiner asks:
👉 *"How model understands text?"*

Answer:

👉

“Model cannot understand text directly, so we convert text into numbers using TF-IDF. Then neural network learns patterns from these numbers.”

🔥 BASIC QUESTIONS (VERY COMMON)

1. What is your project about?

👉

“My project performs sentiment analysis on IMDB movie reviews. It classifies reviews as positive or negative using a neural network after converting text into numerical form using TF-IDF.”

---

2. What is Sentiment Analysis?

👉

“Sentiment analysis is a technique in NLP used to determine whether a text expresses positive, negative, or neutral sentiment.”

---

3. Why did you choose this project?

👉

“This project helps understand Natural Language Processing and real-world applications like product reviews, social media analysis, and customer feedback systems.”

---

4. What dataset did you use?

👉

“I used the IMDB movie review dataset which contains labeled reviews as positive or negative.”

🔥 DATA PREPROCESSING QUESTIONS

5. Why do we clean text data?

👉
"To remove noise like HTML tags, special characters, and convert text into a uniform format so the model can learn better."

---

6. What cleaning steps did you perform?

👉

- Converted text to lowercase
- Removed HTML tags
- Removed special characters

---

7. Why convert text to lowercase?

👉
"To treat words like 'Movie' and 'movie' as the same."

---

8. What is stopword removal? Did you use it?

👉
"Stopwords are common words like 'is', 'the'. I did not explicitly remove them, but TF-IDF reduces their importance automatically."

🔥 TF-IDF QUESTIONS (VERY IMPORTANT)

9. What is TF-IDF?

👉
"TF-IDF converts text into numerical form based on how important a word is in a document compared to the entire dataset."

---

10. Why did you use TF-IDF?

👉
"Because machine learning models cannot understand text directly, so TF-IDF converts text into numerical features."

---

11. What is max_features=5000?

👉
"It limits the model to use only the top 5000 most important words to reduce dimensionality and improve performance."

🔥 MODEL QUESTIONS

12. What model did you use?

👉
"I used a Sequential Neural Network with Dense layers."

---

13. Why neural network?

👉
"Neural networks can capture complex patterns in data and perform well for classification tasks."

---

14. What is Sequential model?

👉
"It is a type of neural network where layers are added one after another in sequence."

---

15. Explain your model architecture.

👉
"Input layer → Hidden layer (128 neurons) → Hidden layer (64 neurons) → Output layer (1 neuron with sigmoid)."

---

16. Why use ReLU activation?

👉
"ReLU helps in faster training and avoids vanishing gradient problem."

---

17. Why use Sigmoid in output?

👉
“Because this is a binary classification problem and sigmoid gives output between 0 and 1.”

---

18. What is binary crossentropy?

👉
“It is a loss function used for binary classification to measure error between predicted and actual values.”

---

19. What is optimizer?

👉
“Optimizer updates model weights to reduce loss. I used Adam optimizer because it is efficient and widely used.”

🔥 TRAINING QUESTIONS

20. What is epoch?

👉
“One complete pass of training data through the model.”

---

21. What is batch size?

👉
“Number of samples processed at once before updating weights.”

---

22. Why train-test split?

👉
“To evaluate model performance on unseen data.”

---

23. Why 80-20 split?

👉
“80% for training ensures good learning and 20% for testing ensures proper evaluation.”

🔥 OUTPUT & PERFORMANCE

24. How do you evaluate your model?

👉
"Using accuracy metric on test data."

---

25. What is accuracy?

👉
"Accuracy is the ratio of correct predictions to total predictions."

---

26. What is overfitting?

👉
"When model performs well on training data but poorly on test data."

---

27. How to avoid overfitting?

👉

- Use more data
- Use dropout
- Reduce model complexity

🔥 PRACTICAL QUESTIONS (EXAMINER FAVORITE)

28. What happens if new words come in test data?

👉
"They are ignored because they are not part of the TF-IDF vocabulary."

---

29. Why convert text to numbers?

👉
"Because machine learning models only work with numerical data."

---

30. Can this model handle neutral sentiment?

👉
"No, currently it only handles binary classification (positive/negative)."

---

31. What are limitations of your model?

👉

- Cannot understand sarcasm
- Limited vocabulary
- No context understanding

---

32. How can you improve this project?

👉

- Use LSTM or Transformer models
- Use word embeddings like Word2Vec
- Increase dataset size

---

33. Difference between ML and DL here?

👉
"ML uses simple models like Logistic Regression, while DL uses neural networks for better feature learning."

🔥 ADVANCED QUESTIONS (HIGH CHANCE)

34. Why not use Bag of Words?

👉
"TF-IDF is better because it gives importance to rare words and reduces impact of common words."

---

35. Why sigmoid threshold 0.5?

👉
"If output > 0.5 → positive, else negative."

---

36. What is feature vector?

👉
"Numerical representation of text after TF-IDF."

---

37. Why use validation data?

👉
"To monitor model performance during training."

🔥 **FINAL MASTER ANSWER (IF EXAMINER SAYS "EXPLAIN FULL FLOW")**

👉
"First, I load and clean the dataset by removing unwanted characters.
Then I convert text into numerical form using TF-IDF.
Next, I split data into training and testing sets.
I build a neural network model and train it on training data.
Finally, I evaluate accuracy using test data and visualize performance using graphs."